Reach, Not Judgement: What Nine Preregistered Studies Show About When Self-Modification Pays Off

Junai P. Felix*
Independent Researcher, Taipei, Taiwan

October 2026
Version 1.0

Abstract

Definitions of recursive self-improvement ask that a system rewrite its own improvement mechanism. They rarely ask whether the rewriting helps, or whether what it helps with generalises. We report nine preregistered studies of one bounded self-improving program-repair system. Each was locked and hash-frozen before its confirmatory run.

A writable generator of self-modifications beat a fixed one (+5.2 percentage points, pp), and a self-extending vocabulary beat the designer's (+4.1 pp). A complete vocabulary supplied by the designer beat both. The system met every clause of a published Level-4 definition. Yet five successive tests found no causal benefit of rewriting the improvement mechanism, and the last found harm (-1.12 pp).

Layer-by-layer ceilings explained why. The rewritable parameters had about 1.7 pp to give; the headroom lay in the policy's form, its observations and its search. When the mechanism's structure became rewritable, one of two campaigns adopted a structural change on its own evidence. The preregistered causal criteria were then met for the first time: +2.57 pp against the default ($p = 0.0028$) and +3.52 pp against random rewriting ($p = 0.0011$).

Across studies, the improvement loop's products transferred to 164 real bugs: +4 to +5 bugs over the starting policy, replicated three times. No rewrite of the mechanism added a significant real-bug gain, and gains on training families largely vanished on held-out families. A post-hoc analysis traces this to the system's observations: no policy form transferred more than about 5 pp to unseen families.

Productive self-modification here was limited by what the system could change, not by how well it judged its changes. Its benefits stayed in-distribution.

Keywords: recursive self-improvement · self-modification · preregistration · negative results · generalisation · program repair

*Correspondence: junai.felixx@gmail.com.

Contents

1 Introduction **3**

2 Background **4**

3 The System and the Programme **5**
- 3.1 Disclosure boundary 5
- 3.2 Roles 5
- 3.3 The nine studies 6

4 Common Methods **7**

5 Writable Generators (v1, v2) **8**

6 Level 4 as Defined, and Five Null Tests **9**

7 External Validity: Real Bugs **11**

8 Locating the Bottleneck **12**

9 Structural Self-Modification (v9) **13**

10 The Generality Limit **14**

11 Discussion **16**

12 Safety and Containment **17**

13 Limitations **18**

14 Reproducibility and Use of AI Assistance **19**

15 Conclusion **20**

A Study Records **23**

1 Introduction

Liu et al.'s Level 4, *bounded recursive self-improvement* [10], asks that a system propose, validate and apply modifications, including modifications to its own improvement mechanism. Systems such as the Darwin Gödel Machine [20] are placed there. The definition does not ask whether rewriting the mechanism *causes* better results than leaving it alone. Nor does it ask whether the results hold outside the distribution the system improved on. Both questions are hard to answer from a single run, and both matter to how such claims should be read.

This paper reports a programme of nine preregistered studies of one system, the Felix Runtime Module (FRM). FRM is a bounded self-improving program-repair module. Every study locked its question, criteria, seeds and analysis, and froze its code by hash, before any confirmatory run. All results are reported, including two studies stopped at their power gate and two clear failures.

Contributions.

1. **A causal record of self-modification.** Six designs tested whether rewriting the improvement mechanism pays off. Five did not; the sixth did. We show why. Productive recursion was limited by the *reach* of self-modification (what it could change), not by its *judgement* (how it validated changes).
2. **A generality record.** Across studies, gains on training families mostly failed to appear on held-out families. A post-hoc analysis attributes the limit to the system's observations, not its learner.
3. **An external-validity record.** The improvement loop's products transfer to real bugs, replicated three times. Rewrites of the mechanism never added a significant real-bug gain.
4. **Methods for honest self-improvement claims.** Power-gated preregistration, crash-only autonomy with fault injection, evidence-free drift controls, and dissection before redesign. We propose that claims of Level 4 report causal and out-of-distribution evidence alongside the definitional clauses.

2 Background

Definitions. Liu et al. [10] grade self-improving agents by what they can modify. At Level 4 a system rewrites "its own improvement mechanism, which makes the improvement mechanism self-referential", and sustains "long-term progress within a bounded domain". The definition is behavioural: it is met when the system does these things. It does not require a comparison with the same system minus the rewriting. A system can therefore meet Level 4 while its self-rewriting is neutral or harmful. This possibility is what the studies below test.

Self-referential improvers. The idea is old. A Gödel machine rewrites any part of itself once it has proved the rewrite useful [17]. Recent practical systems take two routes:

- code generators that improve the scaffolding that improves them [19];
- archives of coding agents that edit their own code and keep improvements validated on benchmarks [20].

Meta-learning [5] and automated machine learning [6, 16] search over learning procedures, usually from outside the learner.

Our setting is deliberately smaller. An explicit, versioned improvement mechanism is edited by a procedure that is part of that mechanism, inside a sandboxed program-repair domain. Small scale is what makes many preregistered, fully-replayed causal tests affordable.

Program repair. Generate-and-validate repair searches candidate edits and checks each against tests [7, 8, 12]. Learned ranking of candidates is established [11]. Real-bug benchmarks include QuixBugs [9] and HumanEvalFix [13]. We use the domain as a test bed for self-improvement, not to compete with repair systems.

Preregistration. We follow preregistration practice [14, 15]. Before any confirmatory evaluation, each study locks:

- the question, arms, seeds and criteria;
- the decision rule and the analysis;
- its code, frozen by hash.

Each study also lists the development runs that shaped it.

3 The System and the Programme

3.1 Disclosure boundary

We describe FRM at the level of roles and interfaces. We withhold the policy's features and vocabulary, how linear revisions are constructed, and the generator. Everything the causal and generality claims rest on is reported:

- arms, seeds and budgets;
- every criterion and result;
- the policy form changed in the last study.

3.2 Roles

- **Task.** A short function from one of 16 parametric families, with one to three injected faults, visible examples and hidden tests. Four families are held out by a hash of each study's freeze.
- **Search (fixed).** A first-improvement hill climb over single-node syntactic edits, with a budget of 12 evaluations per task.
- **Policy.** Ranks candidate edits. Learning a better policy is the improvement the system makes.
- **Generator.** Shapes which policy revisions can be expressed (studies v1 and v2 made it writable).
- **Improvement mechanism (Imp).** An explicit specification of how evidence becomes policy revisions. It contains the procedure that edits Imp (`meta`).
- **Mission.** 16 rounds in v1 and v2, 32 from v3 on. Each round proposes revisions, screens them and promotes at most one through a fixed external audit on fresh tasks.
- **Supervisor.** Runs each mission as a separate process with crash-only resume and hash-chained receipts. A fault-injection suite of 14 groups, and 8 deliberately broken supervisors it must catch, check that autonomy claims are not artefacts of a lenient harness.

3.3 The nine studies

Table 1: The programme. All dates are October 2026. "pp" is the round-32 test solve rate difference unless stated.

Study	Date	Question	Primary result	Verdict
v1	3	Does a writable generator beat a fixed one?	+5.2 pp, 24/24 seeds, $p = 6 \times 10^{-8}$	PASS
v2	3	Does a self-extending vocabulary beat the designer's?	+4.1 pp, 22/24, $p = 5 \times 10^{-7}$	PASS
v3 (Stage 3)	4	Level-4 clauses; does rewriting Imp help?	Clauses met; +0.35 pp, $p = 0.23$	Definitional PASS; causal FAIL
C3T	5	Are evolved mechanisms better starting points?	+0.49 pp, $p = 0.17$	FAIL
v4	6–7	Within-mission validation of rewrites	Pilot -1.04 pp; power 0.00	NO-GO
v5	8–9	Cross-mission validation	Pilot +0.70 pp; power 0.15	NO-GO
v6	10–11	Noise-aware two-stage validation	-1.12 pp, $p = 0.97$	FAIL
v7	10–11	Do self-improved policies repair real bugs?	+4.85 bugs, 23/24, $p = 3 \times 10^{-7}$	PASS
v9	15–16	Does rewriting Imp's *structure* help?	+2.57 pp, $p = 0.0028$; +3.52 pp vs drift	PASS

Between v6 and v9 a series of exploratory development measurements, on development seeds only, located the bottleneck (Section 8). A language-model proposer (v8) was built but never run.

4 Common Methods

Sequence. Every study followed the same order:

1. Register the protocol in a timestamped project document.
2. Freeze the sources: a hash over every file, the configuration and the initial mechanism.
3. Run **Gate 0**: determinism, crash-only resume, headroom, and the autonomy suite on the frozen build.
4. Where a study adapts, run the adaptation once and post its record.
5. Run **Gate 1** where specified, for example a power estimate from a pilot.
6. Run the confirmatory once, on never-used seeds.

Nothing is rerun, and no seed is dropped.

Arms and controls. Causal questions compare paired arms under common random numbers on the same seeds:

- the system with its mechanism rewritten (E);
- a frozen or default mechanism (D);
- and, the most informative control, R: the same proposal process adopting the same number of rewrites at random, regardless of evidence.

E beating R is what shows that the system's own judgement, rather than change itself, produced a gain.

Statistics. Primary tests are exact one-sided Wilcoxon signed-rank tests [18] on 24 per-seed paired differences. Intervals are percentile bootstrap [1] (20,000 resamples, fixed seed). The causal bars, set in Stage 3 and never relaxed, are:

- $\mathsf{E} - \mathsf{D} \geq +1.0$ pp with $p < 0.01$;
- $\mathsf{E} - \mathsf{R} \geq +0.5$ pp with $p < 0.05$.

Integrity. Every mission must reconcile: its receipt chain, spend, weights and final state. Every adaptive campaign must replay from stored mission records to the same decisions. Every study's sources must match its freeze at evaluation. Autonomy criteria require zero interventions.

5 Writable Generators (v1, v2)

The first two studies asked whether the system can usefully rewrite the *space* in which its policy revisions are expressed.

v1. With the generator writable and driven by evidence, missions reached a test solve rate of 32.3%. With a fixed generator they reached 27.1%: +5.2 pp on 24 of 24 seeds, $p = 6 \times 10^{-8}$. Writable generation also beat a generator changed at random (+3.2 pp, 21/24), and the gain held on held-out families (+2.5 pp, $p = 0.0008$).

v2. A generator that extends its own eligible space, composing new elements from its earlier creations, beat the designer's vocabulary by +4.1 pp (22/24, $p = 5 \times 10^{-7}$). Removing the invented elements dropped performance below the designer baseline, so the invention was causal.

The recurring qualification. In both studies a fixed vocabulary that hands the policy everything up front did better:

- v1: the full set of 315 eligible pairs beat the writable generator by 2.0 pp;
- v2: the complete raw closure of 1,167 elements beat the self-extending generator by 2.3 pp.

Self-extension got most of the way with far fewer elements. When the space is small enough to enumerate, though, enumeration wins. In v2 the gain on held-out families was not significant (+0.9 pp, $p = 0.15$).

6 Level 4 as Defined, and Five Null Tests

v3 (Stage 3). With Imp explicit and self-referential, the system met every clause of the Level-4 definition [2]:

- audited promotions in 24/24 missions;
- adopted rewrites of Imp in 21/24;
- rewrites of the editing procedure itself in 17/24;
- test solve rate from 27.4% to 40.1%, with second-half progress of +2.1 pp ($p = 7.6 \times 10^{-6}$);
- zero interventions in 72 missions.

The causal tier failed. Rewriting Imp did not beat a frozen Imp (+0.35 pp, $p = 0.23$) or random adoption (+0.72 pp, $p = 0.079$). A pre-confirmatory power gate had predicted this, and its estimate was posted before the run.

Four redesigns. Each targeted a diagnosed weakness of the previous design (Figure 1):

- **C3T (transplant).** Do mechanisms evolved in Stage 3 make better starting points for fresh missions? +0.49 pp, $p = 0.17$.
- **v4 (within-mission validation).** A variant runs in a shadow branch for a 4-round window and is compared on 300 fresh tasks. The pilot was -1.04 pp and power was 0.00, so the study stopped at Gate 1. The diagnosis: one lucky promotion swings a window by several points, so validation selects luck, and 40% of validations were exact ties.
- **v5 (cross-mission validation).** Variants are judged across 4 whole missions on fresh seeds. The sign turned positive: +0.70 pp against the default and +1.21 pp against drift. Power was 0.15, so the study stopped at Gate 1. Adoption margins were inside panel noise.
- **v6 (noise-aware validation).** A 4-seed screen, then confirmation on 8 never-reused seeds against a margin. Every adopted edit won its confirmation, yet on 24 fresh seeds the evolved mechanisms did *worse*: -1.12 pp against the default and -1.02 pp against drift.

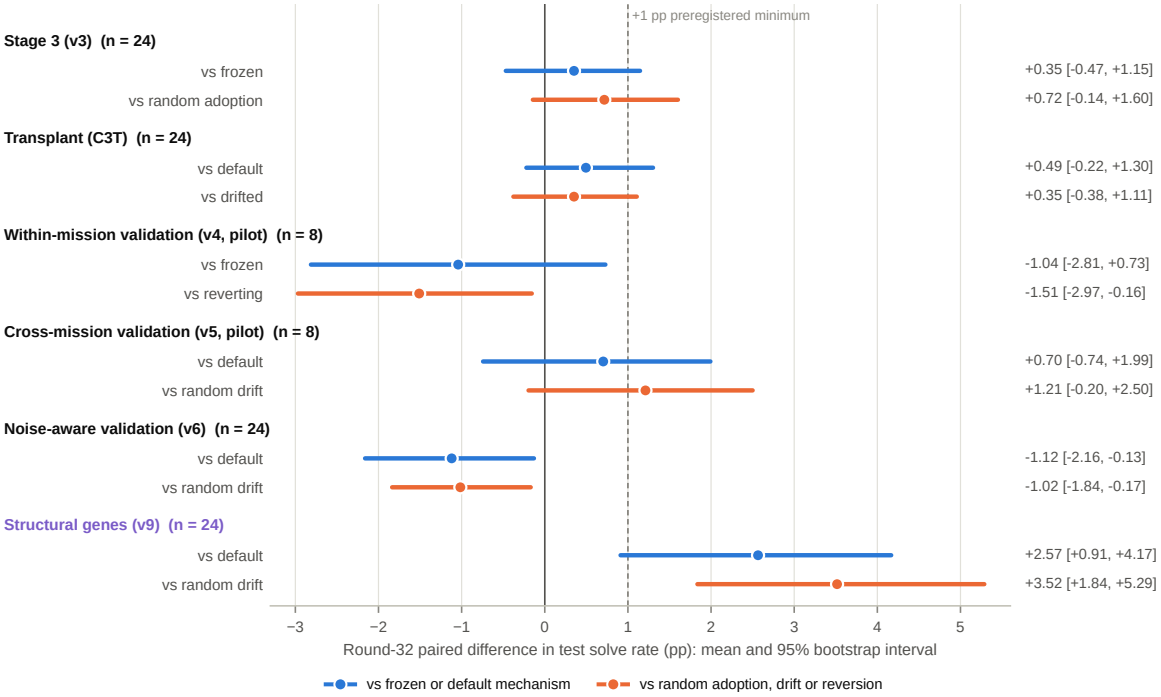


Figure 1: Every preregistered test of whether rewriting the improvement mechanism pays off. Blue: against a frozen or default mechanism. Orange: against random adoption, drift or reversion. v4 and v5 are 8-seed Gate 1 pilots; the others are 24-seed confirmatory runs. Only v9, where the mechanism's structure was rewritable, clears the preregistered bar.

The lesson of v4–v6. Better validation changed the sign of the estimate (v4 to v5) but not its size. Making validation strict (v6) did not rescue it. That pattern points away from the validator and toward what it had to choose from.

7 External Validity: Real Bugs

v7 [4] ran the 72 published Stage 3 end states, unchanged, on 164 valid real bugs (27 from QuixBugs and 137 from HumanEvalFix, in Python). The search was identical, with budget 12. The protocol was locked before any policy was evaluated.

- Self-improved policies solved 25.8 bugs against 21.0 for the starting policy: +4.85 [3.80, 5.84], 23/24 seeds, $p = 3 \times 10^{-7}$.
- The gain disappeared when the learned parameters were permuted.
- It came entirely from HumanEvalFix.

Two later checks reproduced the loop's transfer: the 72 v6 end states (+4.8 to +5.3 bugs by arm) and the 96 v9 end states (+4.2 to +4.6). In every case the mechanism contrast was null (Figure 2).

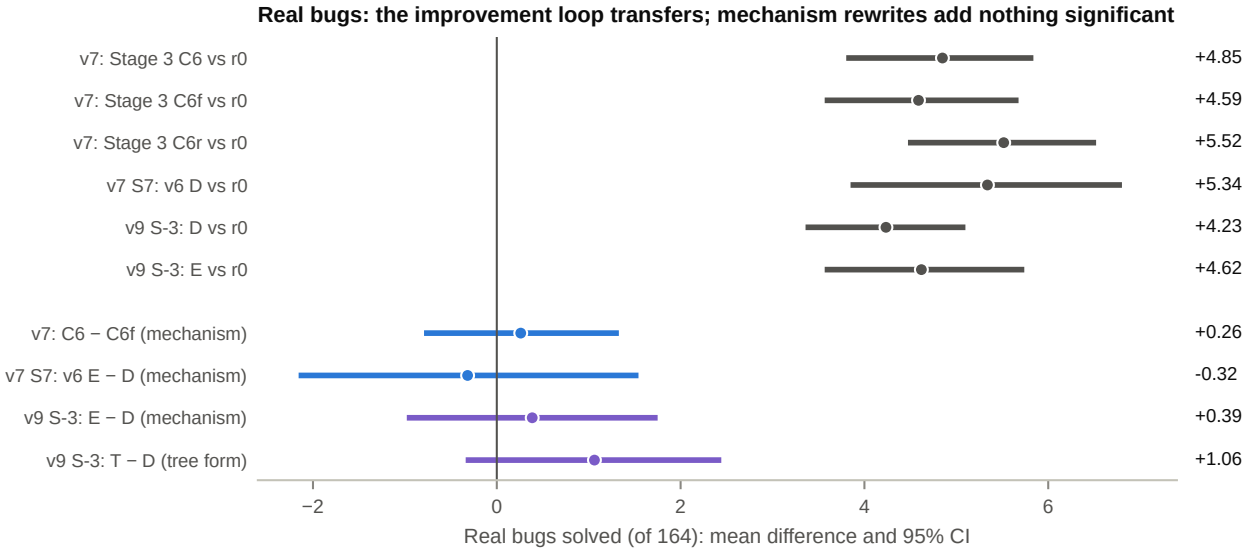


Figure 2: Real bugs, with 95% CIs. Grey: the improvement loop's product against the empty starting policy, in three independent sets of end states. Blue and purple: the contribution of rewriting the mechanism, including v9's structural rewrite and the designer's tree setting. None is significant.

What transfers is the product of the improvement loop. Every arm of every study improves its policy over 32 rounds, and that improvement carries to real bugs. Whether the mechanism was rewritten makes no detectable difference to it.

8 Locating the Bottleneck

After v6 we stopped redesigning the validator and measured where the headroom was. All measurements in this section are exploratory and use development seeds only (Figure 3).

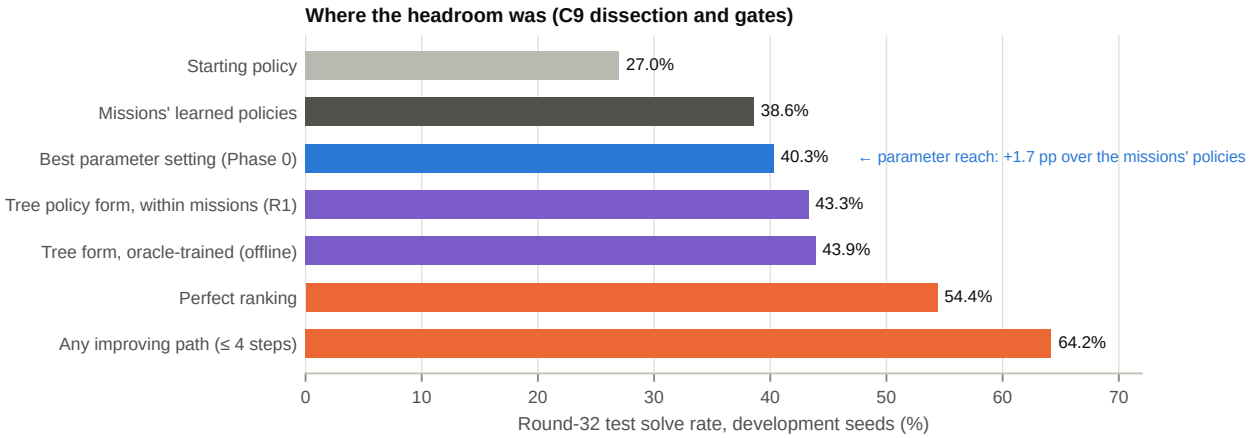


Figure 3: Ceilings on the development test batches (8 seeds, 320 tasks each). Everything the mechanism could rewrite was worth about +1.7 pp over the missions' learned policies. The tree policy form, which the mechanism could not reach, was worth about +4.7 pp within the system's own missions.

Phase 0. We held six mechanism settings fixed for whole missions and compared each with the default. They were the best parameter levers found earlier, alone and combined with one variant, and four structural variants of the linear learner. The best reached +1.68 pp. None met a headroom bar of +3 pp on 6 of 8 seeds. Early estimates regressed toward the mean: one setting stood at +4.5 pp after 3 seeds and +1.7 pp after 8.

Dissection. Offline ceilings showed that the domain was far from saturated. Perfect ranking would solve 54.4%, against 38.6% achieved. The gap sat in three layers outside the mechanism's reach:

- the policy's *form* (linear to nonlinear, about +3–5 pp);
- its observations (about +10 pp);
- the search (about +10 pp).

Gates R0 and R1.

- **R0 (offline).** Execution probes that observe the current program did not help. A tree ensemble beat the best linear policy by +3.36 pp on 8/8 seeds.
- **R1 (within missions, the system's own loop).** Trees combined with an existing budget-split setting gained +4.65 pp on 8/8 seeds.

The diagnosis. The five nulls were not failures of judgement. A perfect validator over the old gene space would have had about 1.7 pp to find, against a +1.0 pp bar and a seed-level standard error of about 0.5 pp. The recursion was limited by its *reach*.

9 Structural Self-Modification (v9)

v9 [3] made the mechanism's structure rewritable.

- **The new gene.** It selects how evidence becomes a policy, from five options: the default linear learner, three linear variants that development had shown to be neutral or harmful, and the tree form.
- **Unchanged from v6.** The proposal operator, the validator, the campaign size (12 generations, 2 campaigns) and the causal bars.
- **A stated risk.** A campaign might never sample the useful option; the probability that both campaigns would sample it was about 0.55.

Campaigns. Campaign 0 adopted the budget split at generation 7 and the tree form at generation 10. The tree form's confirmation margin was +157 tasks against a threshold of 8. The campaign rejected a decoy three times. Its final mechanism equalled the designer's best setting byte for byte. Campaign 1's proposals never drew the tree learner; it adopted two small parameter edits.

Table 2: v9 primary results (pp).

Contrast	Mean	95% CI	$+/-/0$	p
B-1 $\mathsf{E}-\mathsf{D}$	**+2.57**	[0.91, 4.17]	18/6/0	**0.0028**
B-2 $\mathsf{E}-\mathsf{R}$	**+3.52**	[1.84, 5.29]	15/7/2	**0.0011**
$\mathsf{E}-\mathsf{D}$, campaign-0 seeds (structural rewrite)	+5.70	[4.38, 7.03]	12/0/0	0.0002
$\mathsf{E}-\mathsf{D}$, campaign-1 seeds (parameter edits)	-0.57	$[-2.19, 0.89]$	6/6/0	0.66
Designer setting $\mathsf{T}-\mathsf{D}$	+5.68	[4.36, 6.97]	22/2/0	3×10^{-7}

Confirmatory (seeds 7901–7924). Every preregistered criterion was met: Tier A, B-1, B-2, integrity and zero interventions. This is the first causal pass in six tests. The pass comes from one campaign's structural rewrite, chosen by its own validation from a supplied menu. Parameter edits still did nothing. The system captured 45% of the gain the designer's setting gives, because one campaign never sampled the option.

10 The Generality Limit

Across studies. Gains on the families a system trains on often fail to appear on held-out families (Figure 4). v1 is the exception: the writable generator's gain held at about half its size. v2's gain did not reach significance on held-out families. v9's evolved mechanism gained +3.33 pp on training families and +0.26 pp on held-out ones. The designer's tree setting gained +7.62 pp and −0.16 pp.

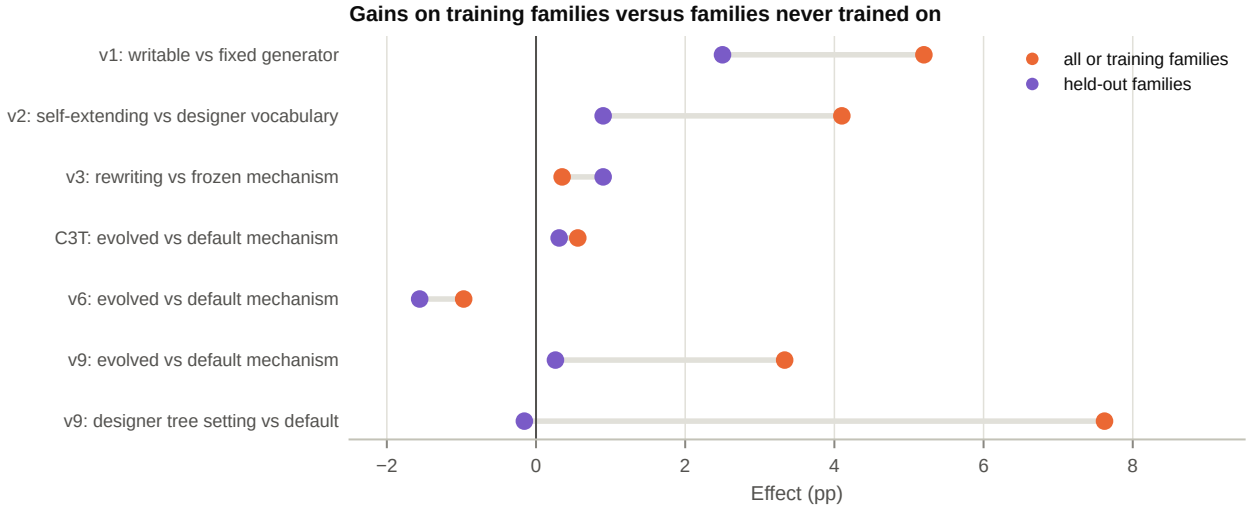


Figure 4: Effect on all or training families (orange) and on held-out families (purple). Sources: each study's preregistered or reported contrasts.

Same families, opposite sign. Gate R1 held out a different set of four families. There the tree form gained +7.2 pp on trained families and lost 3.1 pp on held-out ones. Families held out in R1 but trained in v9 changed sign: dot went from −6.2 to +11.0 pp, and lin2 from −5.0 to +8.1 pp. The tree form learns structure specific to the families it trains on.

Post-hoc: no form transfers more. With oracle-trained policies, four leave-four-families-out folds and 8 development seeds:

- on trained families, forms differ by up to 25 pp;
- on unseen families, every reasonable form reaches 30.6–31.8%, against 26.9% for the empty policy and a ceiling of 54.4% (Figure 5);
- regularising the trees made them worse.

The limit is the system's observations. Its candidate features carry little repair knowledge that transfers across families, so a different learner cannot supply it.

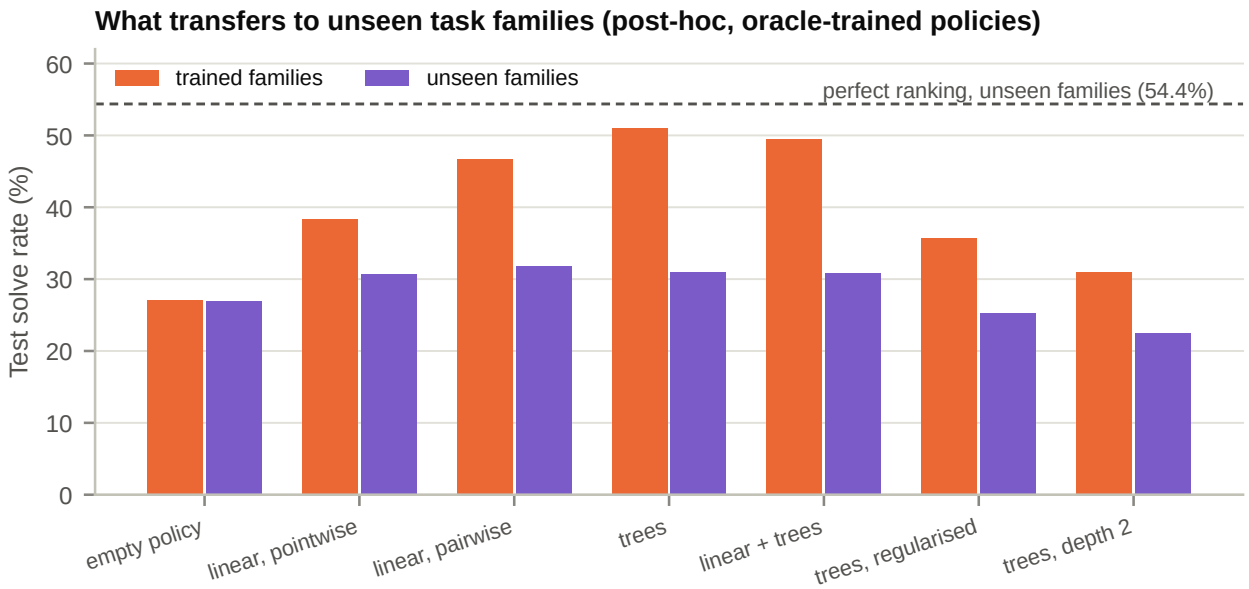


Figure 5: Post-hoc: test solve rate of oracle-trained policies on trained and on unseen task families. The gap to perfect ranking on unseen families (dashed) is not closed by any policy form.

11 Discussion

Reach, not judgement. The programme spent four studies improving how the system judged its own rewrites: transplant, window validation, cross-mission validation and noise-aware confirmation. The estimate moved from negative to positive and back, always within noise. The dissection showed why. Everything the mechanism could change was worth less than the registered bar. Once the mechanism could change its own structure, the same validator found a large gain and adopted it. A self-modifying system can only be as productive as the space it can modify allows. Validation quality matters for the sign of what is adopted, not for how much there is to find.

The controls that mattered. Three design choices separated real effects from apparent ones.

1. **An evidence-free drift arm.** Without it, a pass against a frozen mechanism could reflect change rather than judgement.
2. **Power gates.** They stopped two studies whose pilots could not reach the bar. This saved their confirmatory seeds and prevented two underpowered nulls.
3. **Dissection before redesign.** About 3 minutes of offline ceilings explained what five full studies had not.

Generality. The clearest pattern across nine studies is that in-distribution gains are easy to obtain and hard to carry beyond the training distribution. The improvement loop's products transferred to real bugs; mechanism rewrites did not add to that. Training-family gains mostly vanished on held-out families, and the post-hoc analysis places the limit in what the system observes. A system can improve itself measurably, causally and autonomously, and still be improving only at the distribution it practises on.

Selection and invention. v2's generator invented new compositions, and they were causal, but an enumerating designer still won. v9's mechanism selected a structure it was offered. Neither study shows the system creating a capability its designer did not anticipate. That is a further question, which this programme did not attempt.

What Level-4 claims should report. Behavioural definitions are useful because they can be checked. Our results show they can be met while the defining behaviour does nothing. We suggest that claims of bounded recursive self-improvement report, alongside the definitional clauses:

- a causal contrast against a frozen mechanism *and* against evidence-free rewriting, on fresh seeds;
- an out-of-distribution test, such as held-out task families or an external benchmark;
- preregistration with a power analysis.

By that standard this system reached Level 4 with causal support in one study out of six, and only in-distribution.

12 Safety and Containment

What the system could modify. Self-modification in every study was confined to editing an explicit, bounded specification: generator elements, mechanism parameters and, in v9, a choice among five learners. The system could not modify:

- its own source code;
- the supervisor;
- the audit, the external validator or the test sets;
- its evaluation budget.

It had no network access. Candidate programs ran only to score repair tasks, inside a sandboxed executor with time limits.

What autonomy meant. "Autonomy" here is unattended execution of a bounded experiment. Missions run to completion without human intervention, surviving crashes, hangs, corrupted checkpoints and killed supervisors. The fault suite tests exactly that, and the 8 deliberately broken supervisors check that the suite can fail. It is not open-ended agency: the system chooses nothing outside the specification it edits.

Integrity as a safety property. A self-improving system that can influence its own evaluation can appear to improve while not improving. Here:

- promotion required a fixed external audit on fresh tasks;
- final evaluation used hidden tests, held-out families and, for transfer, external benchmarks;
- every adaptive decision replays from stored records.

Scope. The programme did not pursue code-level or open-ended self-modification. Extensions in that direction would warrant stronger containment and external review than this programme required. Its main finding argues for caution in reading self-improvement claims: measurable, causal self-improvement can coexist with no gain outside the training distribution.

13 Limitations

- **One system, one small domain.** Sixteen families of short functions with injected faults, and a one-step edit search. The real-bug evaluation shows partial external validity, not generality.
- **One author, AI-assisted, self-attested.** The chronology rests on timestamped project documents and hashes. There has been no third-party replication. Audit packages are available to reviewers.
- **Withheld architecture.** Features, vocabulary and generator are not disclosed, which limits independent reimplementation.
- **Sequential design.** Each study was shaped by the previous ones. All such development is disclosed, but the programme as a whole is adaptive. Its causal bars were fixed in Stage 3 and never changed.
- **Definitions.** The Level-4 definition comes from a 2026 preprint that has not been peer reviewed [10].
- **Post-hoc analyses.** The dissection and the generality analysis are exploratory, use development seeds and use oracle-trained policies in part.
- **Infrastructure.** Two studies had host or session interruptions. All were resumed crash-only and disclosed.

14 Reproducibility and Use of AI Assistance

Records. Each study has a registered protocol, a freeze record, a gates record and a results record. Their hashes are listed in Appendix A. Audit packages contain the frozen code, every run record and an auditor script, and are available to reviewers on request under a confidentiality agreement. The script verifies the freeze, replays adaptive decisions and recomputes every reported number.

Author contributions and use of AI assistance. The research direction, the system's original design, and the decisions to proceed, stop, publish and disclose are the author's. The following were carried out with substantial assistance from an AI system (Claude, Anthropic), under the author's direction:

- study design and implementation;
- execution of all runs;
- analysis and figures;
- drafting of this paper.

The AI system was not part of any system under test.

15 Conclusion

Over nine preregistered studies, a bounded self-improving system met a published Level-4 definition. Five tests then showed its self-rewriting did nothing. A dissection showed why: the rewritable part had nothing left to give. Once its structure could be rewritten, the system's own validation found and adopted a change that paid off causally. Its improvements transferred to real bugs. The benefit of rewriting the mechanism did not, and gains on training families did not reach unseen ones.

The results argue for a simple discipline in self-improvement research. Ask not only whether the system rewrites itself, but whether the rewriting causes the gain, against both a frozen and a random rewriter, and whether the gain survives outside the distribution the system practised on.

References

[1] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

[2] Junai P. Felix. Level-4 bounded recursive self-improvement under a published definition: A preregistered test of a self-referential improvement mechanism. Zenodo, record 23138422, 2026.

[3] Junai P. Felix. Rewriting the structure of its own improvement mechanism: A preregistered causal test of bounded recursive self-improvement. Companion preprint, version 1.1, 2026.

[4] Junai P. Felix. Do self-improved repair policies transfer to real bugs? a preregistered external-benchmark test of a level-4 bounded self-improving system. Companion preprint, version 1.1, 2026.

[5] Timothy Hospedales, Antreas Antoniou, Paul Micaelli, and Amos Storkey. Meta-learning in neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(9):5149–5169, 2022.

[6] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren, editors. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.

[7] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

[8] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[9] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. QuixBugs: A multi-lingual program repair benchmark set based on the Quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion)*, pages 55–56, 2017. doi: 10.1145/3135932.3135941.

[10] Shuaiqi Liu, Zhengkai Lin, Yuxiang Zhang, Yuanyi Ren, Yue Wu, Yongbin Li, Zheng Wang, Zhihang Fu, and Jieping Ye. The path to recursive self-improving agents: Foundation, framework, and future directions. Preprints.org, preprint 202608.0051, version 1, 2026. Not peer reviewed.

[11] Fan Long and Martin Rinard. Automatic patch generation by learning correct code. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 298–312, 2016. doi: 10.1145/2837614.2837617.

[12] Martin Monperrus. Automatic software repair: A bibliography. *ACM Computing Surveys*, 51(1): 17:1–17:24, 2018. doi: 10.1145/3105906.

[13] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. OctoPack: Instruction tuning code large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.07124.

[14] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

[15] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[16] Esteban Real, Chen Liang, David R. So, and Quoc V. Le. AutoML-Zero: Evolving machine learning algorithms from scratch. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.

[17] Jürgen Schmidhuber. Gödel machines: Fully self-referential optimal universal self-improvers. In Ben Goertzel and Cassio Pennachin, editors, *Artificial General Intelligence*, pages 199–226. Springer, 2007.

[18] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.

[19] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. In *Conference on Language Modeling (COLM)*, 2024.

[20] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin Gödel machine: Open-ended evolution of self-improving agents. arXiv preprint arXiv:2505.22954, 2025.

A Study Records

Table 3: Freeze and result hashes (SHA-256 prefixes) of the preregistered studies.

Study	Freeze hash	Confirmatory results file
v1	`9099965d...ae428f`	`c660547e...`
v2	`239c23b8...`	`1a153312...`
v3 (Stage 3)	`550193d5...`	`7dc761ec...`
C3T	(recorded in its freeze document)	`169f9ea3...`
v4	`4362e6fe...bc383d`	not run (Gate 1 NO-GO)
v5	`28bced02...a87f`	not run (Gate 1 NO-GO)
v6	`6b1fb796...c5a5`	`8071d7fe...5d6c`
v7	`0b9617b8...aec0`	`f1c95ca7...`
v9	`42288423...dedd`	`a91c9e70...babb`